Guía del notebook `1_10_retropropagacion_base`: retropropagación, un paso

Puntos: 3 · **Tiempo:** 25–30 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_10_retropropagacion_base.ipynb`
Lectura: libro, Retropropagación.

¿Qué vas a aprender?

- Cómo se **reparte la culpa del error** entre las capas de una red: la **retropropagación** (*backpropagation*).
- A calcular el término de error de la capa de salida y el de la capa oculta.
- A calcular los incrementos de **todas** las matrices de pesos.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_10_retropropagacion_base.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El problema

En la Clase 4 aprendimos a corregir los pesos de **una** neurona, porque conocíamos su error (y − ŷ). Pero en una red con capa oculta, **¿cuál es el "error" de una neurona oculta?** Nadie nos dice cuál debió ser su salida.

La solución: mandar el error hacia atrás

1. Calcula el error en la **salida**, como siempre.
2. **Repártelo** hacia la capa oculta: cada neurona oculta recibe una parte proporcional al **peso** que la conecta con la salida (si influyó mucho, carga más culpa).
3. Con esos términos de error, cada capa se actualiza con la misma regla delta.

Capa	Término de error	Incremento de sus pesos
Salida	$\delta_o = (y - \hat{y}) \cdot f'(h_o)$	$\Delta w_ho = \eta \cdot \delta_o \cdot a_h$ (a_h = salida de la oculta)
Oculta	$\delta_h = \delta_o \cdot w_ho \cdot f'(h_h)$	$\Delta w_ih = \eta \cdot \delta_h \cdot x_i$

Con la sigmoide, f′ = salida·(1 − salida).

La red de este notebook

3 entradas → 2 neuronas ocultas → 1 salida, con pesos ya dados.

Paso a paso

Celdas 1–3: funciones y red

Definen la sigmoide, su derivada y la red:

- x = [0.5, 0.1, −0.2], objetivo = 0.6, tasa 0.5;
- pesos de entrada a oculta (3 × 2) y de oculta a salida (2).

Celda: pase frontal (ya está hecha)

Calcula, en este orden:

Valor	Resultado
Entrada a la capa oculta	x·W_ih = [0.24, −0.46]
Salida de la capa oculta	σ(...) ≈ [0.5597, 0.3870]
Entrada a la salida	0.5597·0.1 + 0.3870·(−0.3) ≈ −0.0601
Salida	σ(−0.0601) ≈ 0.4850

Celda `TODO (1 punto)`: términos de error

- `error` = objetivo − salida.
- `del_err_output` = error · salida · (1 − salida).
- `del_err_hidden` = del_err_output · `weights_hidden_output` · salida_oculta · (1 − salida_oculta). Es un vector de 2 números, uno por neurona oculta.

✅ **Comprobación:**

Variable	Valor
error	≈ 0.1150
del_err_output	≈ 0.02873
del_err_hidden	≈ [0.000708, −0.002045]

Celda `TODO (1 punto)`: incrementos y actualización

1. **Tasa:** el notebook pide proponer otra. Para compararte con estos valores, usa primero **0.5**.
2. **Oculta → salida:** `delta_w_h_o` = tasa · del_err_output · salida_oculta.
3. **Entrada → oculta:** `delta_w_i_h` = tasa · del_err_hidden · x, ordenado como **matriz 3 × 2**.
   Pista: `x[:, None]` convierte x en una columna (3 × 1); multiplicada por un vector de 2 da una matriz 3 × 2.
4. **Actualizar:** suma cada incremento a su matriz de pesos.

✅ **Comprobación** (tasa 0.5):

- **Incremento oculta → salida** ≈ [0.00804, 0.00556].
- **Incremento entrada → oculta** ≈

```
[[ 0.000177 -0.000511]
 [ 0.000035 -0.000102]
 [-0.000071  0.000204]]
```

- **Nuevos pesos oculta → salida** ≈ [0.10804, −0.29444].
- Con otra tasa, todos los incrementos se escalan en la misma proporción. Por ejemplo, con 1.0 son el doble.

Pregunta (1 punto)

¿Son del mismo tamaño las matrices de incrementos que las de pesos? Sí: cada peso necesita su propio incremento.

- La de entrada → oculta es **3 × 2**.
- La de oculta → salida tiene **2** elementos.

Puedes comprobarlo con `.shape`.

Para pensar

¿Por qué los incrementos de la capa oculta son tan pequeños? El error llega multiplicado por pesos pequeños (0.1 y −0.3) y por derivadas de la sigmoide (≤ 0.25). En redes con muchas capas, esto se repite y el error casi desaparece: es el problema del **gradiente que se desvanece**.

Errores frecuentes

Síntoma	Causa	Solución
`del_err_hidden` es un solo número	Olvidaste multiplicar por el vector `weights_hidden_output`	Debe dar 2 valores
`ValueError: operands could not be broadcast`	Multiplicaste vectores 3 y 2 sin `x[:, None]`	Convierte x en columna
Valores con el signo contrario	Escribiste salida − objetivo	Es **objetivo − salida**
`TypeError: ... NoneType`	`learnrate` sigue en `None`	Asígnale un número